UNIVERSIDAD NACIONAL DEL ALTIPLANO – PUNO

ESCUELA PROFESIONAL DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA



TRABAJO ACADÉMICO

Estructura de Datos - Capítulo I

DOCENTE: ING. Fred Torres Cruz

ESTUDIANTE: Jhon Franklin Quispe Cutimbo

CURSO: Estructura de Datos

SEMESTRE: III

PUNO - PERÚ

2026

Trabajo Encargado N°2

Visita mi repositorio de GitHub: https://github.com/Kuroneki-1012/Estructura-de-Datos-Cap1/tree/main/Trabajo%20encargado%202

Ejercicio 1: Apilar números enteros

Crear un programa que permita ingresar 5 números enteros y almacenarlos en una pila. Luego, retirar y mostrar los elementos uno por uno hasta dejar la pila vacía. El orden de salida debe permitir comprobar el funcionamiento LIFO. Ejemplo: entrada: 4, 8, 12, 16, 20 → salida: 20, 16, 12, 8, 4. Condición: Use las operaciones push(), top(), pop() y empty().

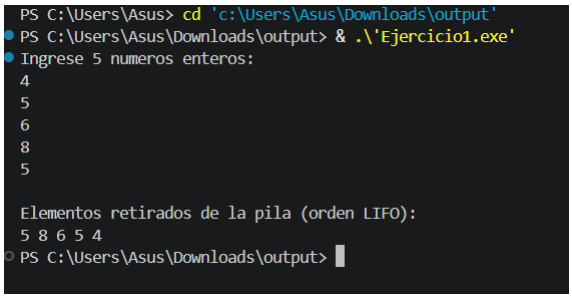
Código

```cpp
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> pila;
    int numero;

    cout << "Ingrese 5 numeros enteros:\n";
    for (int i = 0; i < 5; i++) {
        cin >> numero;
        pila.push(numero);
    }
    cout << "\nElementos retirados de la pila (orden LIFO):\n";
    while (!pila.empty()) {
        cout << pila.top() << " ";
        pila.pop();
    }
    cout << endl;

    return 0;
}
```



Codigo de GitHub: https://github.com/Kuroneki-1012/Estructura-de-Datos-Cap1/tree/431a4b354a2f391f4f411924d378e7c54d3c5f2a/Trabajo%20encargado%202/Ejercicio_1

Ejercicio 2: Invertir una palabra

Leer una palabra ingresada por el usuario. Insertar cada carácter en una pila y, posteriormente, desapilar los caracteres para construir y mostrar la palabra en orden inverso. Ejemplo: DATOS → SOTAD. Condición: Analice por qué una pila permite invertir naturalmente el orden de los elementos.

Código

```cpp
#include <iostream>
#include <stack>
#include <string>
using namespace std;

int main() {
    stack<char> pila;
    string palabra;

    cout << "Ingrese una palabra: ";
    cin >> palabra;

    for (char c : palabra) {
        pila.push(c);
    }

    cout << "Palabra invertida: ";
    while (!pila.empty()) {
        cout << pila.top();
        pila.pop();
    }
    cout << endl;

    return 0;
}
```

```
PS C:\Users\Asus\Downloads\output> cd 'c:\Users\Asus\Downloads\output'
PS C:\Users\Asus\Downloads\output> & .\'Ejercicio2.exe'
Ingrese una palabra: Amor
Palabra invertida: romA
PS C:\Users\Asus\Downloads\output>
```

Codigo de GitHub: https://github.com/Kuroneki-1012/Estructura-de-Datos-Cap1/tree/431a4b354a2f391f4f411924d378e7c54d3c5f2a/Trabajo%20encargado%202/Ejercicio_2

Ejercicio 3: Verificar paréntesis balanceados

Crear un programa que determine si una expresión tiene sus paréntesis correctamente balanceados. Cada vez que aparezca un paréntesis de apertura (, deberá almacenarse en la pila. Cuando aparezca), deberá comprobarse que exista una apertura pendiente. Ejemplos: (a+b)*(c-d) → correcto — ((a+b) → incorrecto. Condición: La expresión será correcta únicamente si nunca se intenta cerrar un paréntesis inexistente y la pila queda vacía al finalizar.

Código

```cpp
#include <iostream>
#include <stack>
#include <string>
using namespace std;

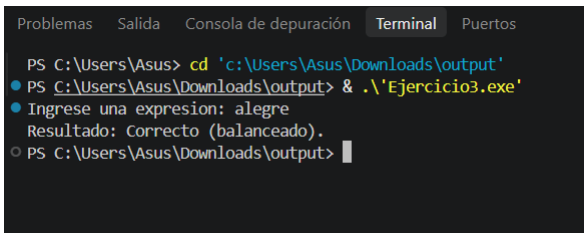
bool estanBalanceados(const string& expresion) {
    stack<char> pila;
    for (char c : expresion) {
        if (c == '(') {
            pila.push(c);
        } else if (c == ')') {
            if (pila.empty()) {
                return false;
            }
            pila.pop();
        }
    }
    return pila.empty();
}

int main() {
    string expresion;
    cout << "Ingrese una expresion: ";
    cin >> expresion;

```

```
    if (estanBalanceados(expresion)) {
        cout << "Resultado: Correcto (balanceado)." << endl;
    } else {
        cout << "Resultado: Incorrecto (no balanceado)." << endl;
    }

    return 0;
}
```



Codigo de GitHub: https://github.com/Kuroneki-1012/Estructura-de-Datos-Cap1/tree/431a4b354a2f391f4f411924d378e7c54d3c5f2a/Trabajo%20encargado%202/Ejercicio_3

Ejercicio 4: Historial de acciones - Deshacer (Undo)

Simular la función Deshacer de un editor. Cada acción realizada por el usuario, por ejemplo .escribir texto", .eliminar palabra.o ïnsertar imagen", deberá almacenarse en una pila. Cuando se solicite deshacer, el programa debe mostrar y eliminar la última acción registrada. Ejemplo: escribir → copiar → eliminar → DESHACER → se revierte .eliminar". Condición: Incluya una opción para mostrar cuál es la acción actualmente ubicada en el tope de la pila.

Código

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

int main() {
    stack<string> historial;
    int opcion;
    string accion;

    do {
        cout << "\n---HISTORIAL DE ACCIONES---\n";
        cout << "1. Realizar nueva accion (ej. escribir, copiar,
            eliminar)\n";
        cout << "2. Deshacer ultima accion (Undo)\n";
```

```
15          cout << "3. Ver la accion ubicada en el tope de la pila\n"
                ;
16          cout << "4. Salir\n";
17          cout << "Seleccione una opcion: ";
18          cin >> opcion;
19          cin.ignore(); //
20
21          switch (opcion) {
22              case 1:
23                  cout << "Ingrese la descripcion de la accion: ";
24                  getline(cin, accion);
25                  historial.push(accion);
26                  cout << "Accion registrada exitosamente.\n";
27                  break;
28
29              case 2:
30                  if (!historial.empty()) {
31                      cout << "DESHACER -> Se revierte \"" <<
                           historial.top() << "\"\n";
32                      historial.pop();
33                  } else {
34                      cout << "El historial esta vacio. No hay
                           acciones para deshacer.\n";
35                  }
36                  break;
37
38              case 3:
39                  if (!historial.empty()) {
40                      cout << "Accion en el tope (ultima realizada):
                            \"" << historial.top() << "\"\n";
41                  } else {
42                      cout << "El historial esta vacio.\n";
43                  }
44                  break;
45
46              case 4:
47                  cout << "Saliendo del programa...\n";
48                  break;
49
50              default:
51                  cout << "Opcion no valida. Intente de nuevo.\n";
52          }
53      } while (opcion != 4);
54
55      return 0;
56  }
```

```
PS C:\Users\Asus\Downloads\output> & .\'Ejercicio4.exe'

--- HISTORIAL DE ACCIONES (UNDO) ---
1. Realizar nueva accion (ej. escribir, copiar, eliminar)
2. Deshacer ultima accion (Undo)
3. Ver la accion ubicada en el tope de la pila
4. Salir
Seleccione una opcion: 1
Ingrese la descripcion de la accion: hola,comer,correr,dormir
Accion registrada exitosamente.

--- HISTORIAL DE ACCIONES (UNDO) ---
1. Realizar nueva accion (ej. escribir, copiar, eliminar)
2. Deshacer ultima accion (Undo)
3. Ver la accion ubicada en el tope de la pila
4. Salir
Seleccione una opcion: 2
DESHACER -> Se revierte "hola,comer,correr,dormir"

--- HISTORIAL DE ACCIONES (UNDO) ---
1. Realizar nueva accion (ej. escribir, copiar, eliminar)
2. Deshacer ultima accion (Undo)
3. Ver la accion ubicada en el tope de la pila
4. Salir
Seleccione una opcion: 3
El historial esta vacio.

--- HISTORIAL DE ACCIONES (UNDO) ---
1. Realizar nueva accion (ej. escribir, copiar, eliminar)
2. Deshacer ultima accion (Undo)
3. Ver la accion ubicada en el tope de la pila
4. Salir
Seleccione una opcion:
```

Codigo de GitHub: https://github.com/Kuroneki-1012/Estructura-de-Datos-Cap1/tree/431a4b354a2f391f4f411924d378e7c54d3c5f2a/Trabajo%20encargado%202/Ejercicio_4

Ejercicio 5: Conversión de decimal a binario

Leer un número entero decimal positivo y convertirlo a binario utilizando una pila. Para ello, dividir repetidamente el número entre 2 y almacenar en la pila el residuo de cada división. Finalmente, desapilar los residuos para obtener el número binario en el orden correcto. Ejemplo: 13 → residuos apilados: 1, 0, 1, 1 → binario: 1101. Condición: No utilice funciones directas de conversión a binario; la solución debe evidenciar el uso de la pila.

Código

```cpp
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> residuos;
    int numero;

    cout << "Ingrese un numero entero decimal positivo: ";
    cin >> numero;

```

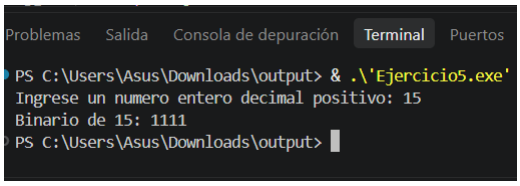
```
    if (numero < 0) {
        cout << "El numero debe ser positivo." << endl;
        return 0;
    }

    if (numero == 0) {
        cout << "Binario: 0" << endl;
        return 0;
    }

    int temp = numero;
    while (temp > 0) {
        residuos.push(temp % 2);
        temp /= 2;
    }

    cout << "Binario de " << numero << ": ";
    while (!residuos.empty()) {
        cout << residuos.top();
        residuos.pop();
    }
    cout << endl;

    return 0;
}
```



Codigo de GitHub: https://github.com/Kuroneki-1012/Estructura-de-Datos-Cap1/tree/431a4b354a2f391f4f411924d378e7c54d3c5f2a/Trabajo%20encargado%202/Ejercicio_5